KIẾN TRÚC MÁY TÍNH

ÔN TẬP GIỮA HỌC KỲ

Ôn tập các nội dung lý thuyết trong nửa đầu của học kỳ bao gồmMáy tính, các khái niệm và công nghệ, Hiệu suất, Kiến trúc tập lệnh. Đồng thời giải các bài tập liên quan đến các nội dung lý thuyết

PHAN ĐÌNH DUY

MỤC TIÊU

1. Phát biểu được các nội dung lý thuyết đã học trong nửa học kỳ đầu
2. Hiện thực được các bài tập liên quan đến kiến trúc tập lệnh

NỘI DUNG

1. Thông tin đề thi
2. Nội dung ôn tập

Thông tin đề thi

Thành phần đánh giá	Tỷ lệ (%)
A1. Quá trình	30%
A2. Giữa kỳ	**20%**
A3. Thực hành	0%
A4. Cuối kỳ	50%

Thông tin đề thi (tt)

- Thời gian làm bài 65 phút

- Ngày thi: 24/4/2025 thứ 5 ca 3 13h00

Thông tin đề thi (tt)

- Tự luận: (1 điểm)
  - Kiến trúc tập lệnh
- Trắc nghiệm: (9 điểm)
  - 18 câu hỏi (0.5 điểm/1 câu)
  - Lý thuyết
  - Bài tập
  - Tất cả các nội dung đã học trong chương 1 và chương 2
    - Máy tính, các khái niệm và công nghệ
    - Hiệu suất
    - Kiến trúc tập lệnh

Nội dung ôn thi

- **Chương 1: Những khái niệm và công nghệ máy tính (Tuần 1)**
- **Chương 2: Kiến trúc bộ lệnh (Tuần 2, 3, 4)**
- Chương 3: Các phép toán số học trong máy tính (Tuần 5, 6)
- Chương 4: Bộ xử lý (Tuần 7, 8)
- Ôn tập giữa kỳ (Tuần 9)
- Chương 5: Bộ xử lý pipeline (Tuần 10)
- Ôn tập (Tuần 11)

CHƯƠNG 1: MÁY TÍNH CÁC KHÁI NIỆM VÀ CÔNG NGHỆ

1. Lịch sử phát triển của máy tính.
2. Phân loại máy tính.
3. Các lớp thực thi bên trong máy tính.
4. Các chức năng và thành phần cơ bản của máy tính.
5. Hiệu suất máy tính

1. Lịch sử phát triển của máy tính

- Thế hệ 0: Máy chuyển động cơ học
- Thế hệ 1: Máy tính sử dụng đèn chân không
- Thế hệ 2: Máy tính sử dụng transistor
- Thế hệ 3: Máy tính sử dụng công nghệ mạch tích hợp
- Thế hệ 4: Máy tính sử dụng công nghệ VLSI

  ⇨Thế hệ 5: Máy tính lượng tử

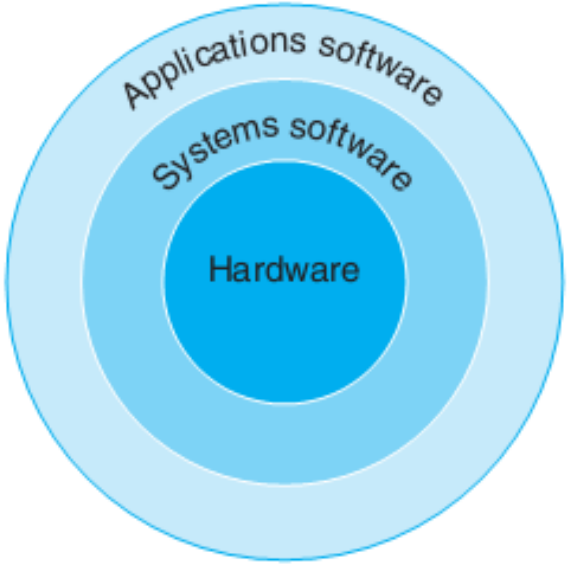
2. Phân loại máy tính

Máy tính được sử dụng trong 3 lớp ứng dụng chính:

- Máy tính cá nhân (Personal computers)
- Máy chủ (Servers)
- Máy tính nhúng (Embedded computers)

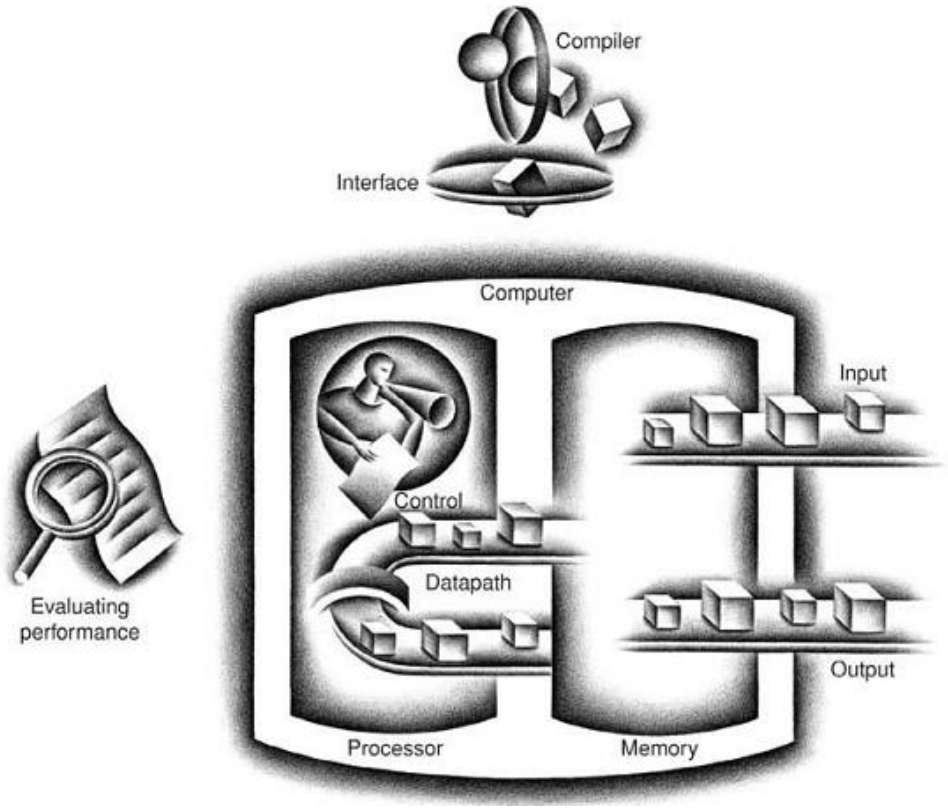
3. Các lớp thực thi bên trong máy tính

- Phần mềm ứng dụng
  - Được viết bằng ngôn ngữ cấp cao (HLL)
- Phần mềm hệ thống
  - Trình biên dịch: chuyển từ mã HLL sang mã máy
  - Hệ điều hành: Microsoft, Linux, MacOS
    - Xử lý nhập/xuất: bàn phím, màn hình, …
    - Quản lý bộ nhớ
    - Lập lịch và chia sẻ tài nguyên cho nhiều chương trình chạy cùng lúc
- Phần cứng
  - Bộ xử lý, Bộ nhớ, Các bộ điều khiển I/O



4. Các thành phần cơ bản của máy tính

- Ngõ nhập (input) ghi dữ liệu vào bộ nhớ và ngõ xuất (output) đọc dữ liệu ra từ bộ nhớ.
- Bộ nhớ (Memory): Lưu trữ chương trình đang chạy và dữ liệu cần bởi chương trình đó.
- Bộ xử lý (Processor): Nhận lệnh và dữ liệu từ bộ nhớ để xử lý.



Ảnh màu

- Để biểu diễn được ảnh màu trên màn hình máy tính, ảnh và màn hình đều được chia thành các hàng và cột, tạo thành một ma trận các ô. Mỗi ô như vậy gọi là 1 pixel.
  - Pixel: Phần tử ảnh nhỏ nhất. Màn hình bao gồm hàng trăm, hoặc ngàn, hoặc triệu pixel được tổ chức thành một ma trận.
- Tùy vào kích cỡ màn hình và độ phân giải (resolution), ma trận hiển thị có thể có độ lớn từ 640×480 đến 3840×2160 (4k) hoặc 7680×4320 (8k) pixels
- Ví dụ: Màn hình với độ phân giải Full HD 1920×1080, tức độ rộng nó 1920 (1920 cột) và cao là 1080 (1080 hàng), và có tổng cộng 1920×1080 = 2073600 pixels tồn tại trên màn hình, hay 2.0736 Megapixels (1 Megapixels = 1 triệu pixels).

5. Hiệu suất

- Thời gian thực thi và hiệu suất
- Clock
- CPI
- Tính hiệu suất theo IPS/MIPS
- Các kỹ thuật nâng cao hiệu suất

5.3. CPI (Clock cycle Per Instruction)

Tổng số chu kỳ clock = Tổng số lệnh * CPI

Thời gian thực thi = Tổng số lệnh * CPI * Chu kỳ clock

$$\text{Thời gian thực thi} = \frac{\text{Tổng số lệnh} * \text{CPI}}{\text{Tần số clock}}$$

5.3. CPI (tt)

- Mỗi tập lệnh có nhiều nhóm lệnh khác nhau

$$\text{Chu kỳ clock} = \sum_{i=1}^{n} (\text{CPI}_i * \text{Số lệnh trong nhóm}_i)$$

$$\text{CPI} = \frac{\text{chu kỳ clock}}{\text{Tổng số lệnh}} = \sum_{i=1}^{n} \left(\text{CPI}_i * \frac{\text{Số lệnh trong nhóm}_i}{\text{Tổng số lệnh}} \right)$$

5.4. Tính hiệu suất theo IPS/MIPS

- MIPS (Million instructions per second): Một cách đo tốc độ thực thi của chương trình dựa trên số lượng triệu lệnh trên giây. MIPS được tính bằng số lượng lệnh chia cho tích của thời gian thực thi và giá trị 106.

$$\text{MIPS} = \frac{\text{Number of instructions}}{\text{Execution time} \times 10^6}$$

$$\text{MIPS} = \frac{\text{Number of instructions}}{\frac{\text{Number of instructions} \times \text{CPI}}{\textit{Clock rate}} \times 10^6} = \frac{\text{Clock rate}}{\text{CPI} \times 10^6}$$

- IPS (Instructions per second): Số lệnh trên giây. Nếu đề bài chỉ yêu cầu tính IPS thì không cần chia cho 10^6

Bài tập chương 1

- Sử dụng file bài tập chương 1

CHƯƠNG 2: KIẾN TRÚC TẬP LỆNH

1. Giới thiệu (kiến trúc bộ lệnh là gì, có những kiến trúc nào)
2. Các phép tính (các lệnh cơ bản trong kiến trúc tập lệnh MIPS)
3. Toán hạng (3 loại toán hạng: thanh ghi, bộ nhớ và hằng số)
4. Số có dấu và không dấu (biểu diễn số có dấu và không dấu trên máy tính)
5. Biểu diễn lệnh (chuyển lệnh ASM ↔ mã máy)
6. Các phép tính Logic (nhắc lại về các lệnh logic)
7. Các lệnh điều kiện và nhảy (chuyển các câu lệnh rẽ nhánh như: if, for, while,…)

1. Giới thiệu - Tập lệnh MIPS

- Thiết kế theo kiến trúc thanh ghi - thanh ghi
- Độ rộng lệnh: Cố định 32-bit cho tất cả các lệnh
- Định dạng lệnh: R, I, J
- Tập thanh ghi: 32 thanh ghi 32 bit, thanh ghi $zero luôn bằng 0
- Kiểu dữ liệu: Byte (8 bit), halfword (16 bit), word (32 bit)
- Chế độ định địa chỉ: 5 chế độ
- Toán hạng: Thanh ghi, số tức thời (bù 2), bộ nhớ
- Định địa chỉ theo byte

2. Các phép tính - Các lệnh trong MIPS (tt)

Loại	Lệnh	Ví dụ	Ý nghĩa	ĐD
Số học	Cộng	add $s1,$s2,$s3	$s1=$s2 + $s3	R
	Trừ	sub $s1,$s2,$s3	$s1=$s2 - $s3	R
	Cộng tức thì	addi $s1,$s2,20	$s1=$s2 + 20	I
Truyền dữ liệu	Nạp word	lw $s1,20($s2)	$s1= Mem[$s2 + 20]	I
	Lưu word	sw $s1,20($s2)	Mem[s2 + 20]=$s1	I
Luận lý	NOR luận lý	nor $s1,$s2,$s3	$s1= ~($s2 | $s3)	R
	Dịch phải luận lý	srl $s1,$s2,10	$s1=$s2>>10	R
Rẽ nhánh	Nhảy nếu bằng	beq $s1,$s2, label	Nếu ($s1==$s2) đi đến label	I
Nhảy	Nhảy	j label	Đi đến label	J

3. Toán hạng

Toán hạng là một dữ liệu được dùng để tính toán

MIPS có 3 loại toán hạng:

- Toán hạng thanh ghi: Dữ liệu nằm trong thanh ghi — `add $t0, $s1, $s2`
- Toán hạng bộ nhớ: Dữ liệu nằm trong bộ nhớ — `lw $t0, -8($s2)`
- Toán hạng số tức thời: Dữ liệu nằm ngay trong lệnh — `andi $t0, $s2, -8`

4. Số có dấu và không dấu (tt)

Khi làm việc với các lệnh của MIPS, lưu ý:

- Mở rộng có dấu (Sign-extend)
- Mở rộng không dấu (Zero-extend)

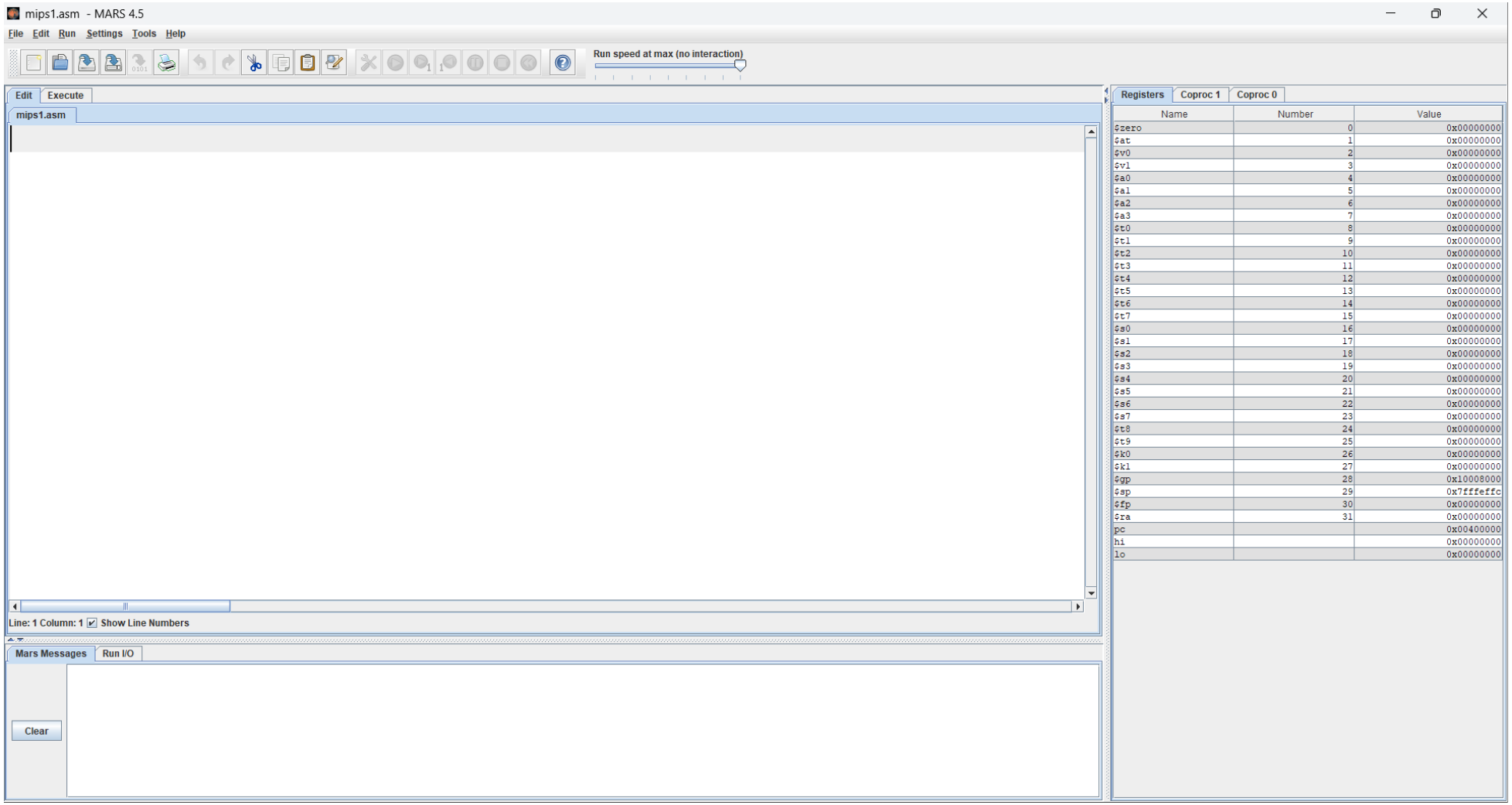
5. Các phép tính Logic

Logical operations	C operators	Java operators	MIPS instructions
Shift left	<<	<<	sll
Shift right	>>	>>>	srl
Bit-by-bit AND	&	&	and, andi
Bit-by-bit OR	|	|	or, ori

C và Java các phép tính logic và lệnh MIPS tương ứng.

- Shift: Lệnh dịch chuyển bit.
- AND: là phép toán logic “VÀ”.
- OR: là một phép toán logic “HOẶC”
- NOR: NOT OR.
- Hằng số rất hữu ích trong các phép toán logic AND và OR cũng như trong phép tính số học, vì vậy MIPS cung cấp các lệnh trực tiếp andi và ori.

6. Lập trình ASM trên MARS 4.5 (tt)



7. Biểu diễn lệnh

- Lệnh được biểu diễn bằng các mã nhị phân (mã máy)
- Định dạng lệnh là một hình thức biểu diễn một lệnh dưới dạng các trường mã nhị phân
- Lệnh của MIPS đều rộng 32 bit
- MIPS có 3 định dạng lệnh:
  - Định dạng lệnh R: Cho các thao tác tuần túy trên thanh ghi
  - Định dạng lệnh I: Cho các thao tác sử dụng số tức thời có giá trị nhỏ và vừa
  - Định dạng lệnh J: Cho các thao tác sử dụng số tức thời có giá trị lớn

Chuyển đổi lệnh ASM MIPS sang mã máy

- Chuyển câu lệnh assembly (ASM) MIPS sau sang mã máy:

  and $t3, $s0, $s2

- Thực hiện việc chuyển đổi theo các bước sau:
  - Bước 1: Tra bảng "MIPS reference data" xem lệnh *and* thuộc định dạng nào => R type

And	and	R	R[rd] = R[rs] & R[rt]	$0 / 24_{hex}$

op	rs	rt	rd	shamt	funct

Bảng "MIPS reference data"

oration to separate card 2. Fold bottom side (columns 3 and 4) together

MIPS Reference Data ①

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	0 / 20_{hex}
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		0 / 21_{hex}
And	and	R	R[rd] = R[rs] & R[rt]		0 / 24_{hex}
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3)	c_{hex}
Branch On Equal	beq	I	if(R[rs]==R[rt]) PC=PC+4+BranchAddr	(4)	4_{hex}
Branch On Not Equal	bne	I	if(R[rs]!=R[rt]) PC=PC+4+BranchAddr	(4)	5_{hex}
Jump	j	J	PC=JumpAddr	(5)	2_{hex}
Jump And Link	jal	J	R[31]=PC+8;PC=JumpAddr	(5)	3_{hex}
Jump Register	jr	R	PC=R[rs]		0 / 08_{hex}
Load Byte Unsigned	lbu	I	R[rt]={24'b0,M[R[rs] +SignExtImm](7:0)}	(2)	24_{hex}
Load Halfword Unsigned	lhu	I	R[rt]={16'b0,M[R[rs] +SignExtImm](15:0)}	(2)	25_{hex}
Load Linked	ll	I	R[rt] = M[R[rs]+SignExtImm]	(2,7)	30_{hex}
Load Upper Imm.	lui	I	R[rt] = {imm, 16'b0}		f_{hex}
Load Word	lw	I	R[rt] = M[R[rs]+SignExtImm]	(2)	23_{hex}
Nor	nor	R	R[rd] = ~ (R[rs] | R[rt])		0 / 27_{hex}
Or	or	R	R[rd] = R[rs] | R[rt]		0 / 25_{hex}
Or Immediate	ori	I	R[rt] = R[rs] | ZeroExtImm	(3)	d_{hex}
Set Less Than	slt	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0		0 / $2a_{hex}$

ARITHMETIC CORE INSTRUCTION SET ②

NAME, MNEMONIC		FOR-MAT	OPERATION		OPCODE / FMT /FT / FUNCT (Hex)
Branch On FP True	bc1t	FI	if(FPcond)PC=PC+4+BranchAddr	(4)	11/8/1/--
Branch On FP False	bc1f	FI	if(!FPcond)PC=PC+4+BranchAddr	(4)	11/8/0/--
Divide	div	R	Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]		0/--/--/1a
Divide Unsigned	divu	R	Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]	(6)	0/--/--/1b
FP Add Single	add.s	FR	F[fd]= F[fs] + F[ft]		11/10/--/0
FP Add Double	add.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} + {F[ft],F[ft+1]}		11/11/--/0
FP Compare Single	c.*x*.s*	FR	FPcond = (F[fs] *op* F[ft]) ? 1 : 0		11/10/--/*y*
FP Compare Double	c.*x*.d*	FR	FPcond = ({F[fs],F[fs+1]} *op* {F[ft],F[ft+1]}) ? 1 : 0		11/11/--/*y*
* (*x* is eq, lt, or le) (*op* is ==, <, or <=) (*y* is 32, 3c, or 3e)					
FP Divide Single	div.s	FR	F[fd] = F[fs] / F[ft]		11/10/--/3
FP Divide Double	div.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} / {F[ft],F[ft+1]}		11/11/--/3
FP Multiply Single	mul.s	FR	F[fd] = F[fs] * F[ft]		11/10/--/2
FP Multiply Double	mul.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} * {F[ft],F[ft+1]}		11/11/--/2
FP Subtract Single	sub.s	FR	F[fd]=F[fs] - F[ft]		11/10/--/1
FP Subtract Double	sub.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} - {F[ft],F[ft+1]}		11/11/--/1
Load FP Single	lwc1	I	F[rt]=M[R[rs]+SignExtImm]	(2)	31/--/--/--
Load FP Double	ldc1	I	F[rt]=M[R[rs]+SignExtImm]; F[rt+1]=M[R[rs]+SignExtImm+4]	(2)	35/--/--/--
Move From Hi	mfhi	R	R[rd] = Hi		0 /--/--/10
Move From Lo	mflo	R	R[rd] = Lo		0 /--/--/12
Move From Control	mfc0	R	R[rd] = CR[rs]		10 /0/--/0
Multiply	mult	R	{Hi,Lo} = R[rs] * R[rt]		0/--/--/18
Multiply Unsigned	multu	R	{Hi,Lo} = R[rs] * R[rt]	(6)	0/--/--/19
Shift Right Arith.	sra	R	R[rd] = R[rt] >>> shamt		0/--/--/3
Store FP Single	swc1	I	M[R[rs]+SignExtImm] = F[rt]	(2)	39/--/--/--
Store FP Double	sdc1	I	M[R[rs]+SignExtImm] = F[rt]; M[R[rs]+SignExtImm+4] = F[rt+1]	(2)	3d/--/--/--

Bảng "MIPS reference data"

. Fold bottom side (columns 3 and 4) together

MIPS Reference Data ①

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	$0 / 20_{hex}$
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		$0 / 21_{hex}$
And	and	R	R[rd] = R[rs] & R[rt]		$0 / 24_{hex}$
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3)	c_{hex}
Branch On Equal	beq	I	if(R[rs]==R[rt]) PC=PC+4+BranchAddr	(4)	4_{hex}
Branch On Not Equal	bne	I	if(R[rs]!=R[rt]) PC=PC+4+BranchAddr	(4)	5_{hex}
Jump	j	J	PC=JumpAddr	(5)	2_{hex}
Jump And Link	jal	J	R[31]=PC+8;PC=JumpAddr	(5)	3_{hex}
Jump Register	jr	R	PC=R[rs]		$0 / 08_{hex}$

Chuyển đổi lệnh ASM MIPS sang mã máy (tt)

- Chuyển câu lệnh assembly (ASM) MIPS sau sang mã máy:

and $t3, $s0, $s2

- Thực hiện việc chuyển đổi theo các bước sau:
  - Bước 1: Tra bảng "MIPS reference data" xem lệnh *and* thuộc định dạng nào => R type
  - Bước 2: Tra các trường opcode và function (nếu có)

And	and	R	R[rd] = R[rs] & R[rt]	$0 / 24_{hex}$

op	rs	rt	rd	shamt	funct
000000					100100

Chuyển đổi lệnh ASM MIPS sang mã máy (tt)

- Chuyển câu lệnh assembly (ASM) MIPS sau sang mã máy:

and $t3, $s0, $s2 => $t3 = $s0 & $s2

- Thực hiện việc chuyển đổi theo các bước sau:
  - Bước 1: Tra bảng "MIPS reference data" xem lệnh *and* thuộc định dạng nào => R type
  - Bước 2: Tra các trường opcode và function (nếu có)
  - Bước 3: Tra vị trí và chỉ số các thanh ghi

NAME	NUMBER
$zero	0
$at	1
$v0-$v1	2-3
$a0-$a3	4-7
$t0-$t7	8-15
$s0-$s7	16-23
$t8-$t9	24-25
$k0-$k1	26-27
$gp	28
$sp	29
$fp	30
$ra	31

And and R R[rd] = R[rs] & R[rt] $0 / 24_{hex}$

op	rs	rt	rd	shamt	funct
000000	10000	10010	01011		100100

Chuyển đổi lệnh ASM MIPS sang mã máy (tt)

- Chuyển câu lệnh assembly (ASM) MIPS sau sang mã máy:

and $t3, $s0, $s2 => $t3 = $s0 & $s2

- Thực hiện việc chuyển đổi theo các bước sau:
  - Bước 1: Tra bảng "MIPS reference data" xem lệnh *and* thuộc định dạng nào => R type
  - Bước 2: Tra các trường opcode và function (nếu có)
  - Bước 3: Tra vị trí và chỉ số các thanh ghi
  - Bước 4: Điền trường shamt và hoàn thành mã máy của lệnh

op	rs	rt	rd	shamt	funct
000000	10000	10010	01011	00000	100100

000000100001001001011000001001000 ⇨ 0x02125824

Chuyển mã máy sang ngôn ngữ Assembly (ASM) MIPS

- Chuyển mã máy sau sang câu lệnh assembly MIPS:

0x01304024

- Thực hiện việc chuyển đổi theo các bước sau:
  - Bước 1: Chuyển mã máy sang hệ nhị phân
  - Bước 2: Chọn 6bit đầu và tra opcode, nếu opcode bằng 000000 thì tra tiếp function 6bit cuối trong bảng MIPS

0x	0	1	3	0	4	0	2	4
	0000	0001	0011	0000	0100	0000	0010	0100

op		funct
000000	01 0011 0000 0100 0000 00	100100

Từ đó suy ra lệnh "and" và thuộc format "R"

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	$0 / 20_{hex}$
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		$0 / 21_{hex}$
And	and	R	R[rd] = R[rs] & R[rt]		$0 / 24_{hex}$

Chuyển mã máy sang ngôn ngữ Assembly MIPS (tt)

- Chuyển mã máy sau sang câu lệnh assembly MIPS:

0x01304024

- Thực hiện việc chuyển đổi theo các bước sau:
  - Bước 2: Chọn 6bit đầu và tra opcode, nếu opcode bằng 000000 thì tra tiếp function 6bit cuối trong bảng MIPS
  - Bước 3: Tìm các thanh ghi tương ứng với các trường trong format R

op		funct
000000	01 0011 0000 0100 0000 00	100100

op	Rs	rt	rd	shamt	funct
000000	01001	10000	01000	00000	100100
	9 ⇨ t1	16 ⇨ s0	8 ⇨ t0		

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	$0 / 20_{hex}$
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		$0 / 21_{hex}$
And	and	R	R[rd] = R[rs] & R[rt]		$0 / 24_{hex}$
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3)	c_{hex}

Chuyển mã máy sang ngôn ngữ Assembly MIPS (tt)

- Chuyển mã máy sau sang câu lệnh assembly MIPS:

0x01304024

- Thực hiện việc chuyển đổi theo các bước sau:
  - Bước 2: Chọn 6bit đầu và tra opcode, nếu opcode bằng 000000 thì tra tiếp function 6bit cuối trong bảng MIPS
  - Bước 3: Tìm các thanh ghi tương ứng với các trường trong format R
  - Bước 4: Hoàn thành lệnh: and $t0, $t1, $s0

op	Rs	rt	rd	shamt	funct
000000	01001	10000	01000	00000	100100
	9 ⇨ t1	16 ⇨ s0	8 ⇨ t0		

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	$0 / 20_{hex}$
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		$0 / 21_{hex}$
And	and	R	R[rd] = R[rs] & R[rt]		$0 / 24_{hex}$
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3)	c_{hex}

8. Các lệnh điều kiện và nhảy (tt)

- Các lệnh rẽ nhánh có điều kiện (conditional branch) của MIPS:

Conditional branch	branch on equal	`beq $s1,$s2,25`	if ($s1 == $s2) go to PC + 4 + 100
	branch on not equal	`bne $s1,$s2,25`	if ($s1!= $s2) go to PC + 4 + 100
	set on less than	`slt $s1,$s2,$s3`	if ($s2 < $s3) $s1 = 1; else $s1 = 0
	set on less than unsigned	`sltu $s1,$s2,$s3`	if ($s2 < $s3) $s1 = 1; else $s1 = 0
	set less than immediate	`slti $s1,$s2,20`	if ($s2 < 20) $s1 = 1; else $s1 = 0
	set less than immediate unsigned	`sltiu $s1,$s2,20`	if ($s2 < 20) $s1 = 1; else $s1 = 0

Bài tập chương 2

- Sử dụng file bài tập chương 2

Tóm tắt lại nội dung buổi học

- Thông tin đề thi
- Nội dung ôn tập
  - Chương 1
  - Chương 2

THẢO LUẬN

